SceneFork turns passive video footage into a live, interactive 2.5D physics and audio sandbox in the browser. To win both the Grand Prize and the **$20k Most Viral Project** bonus, the system needs to feel instantaneous during a live 3-minute demo while demonstrating deep, closed-loop orchestration across Meta's entire model stack.

1. The Core Technical Trick: "Freeze, Clean-Plate, and Fork"

The hardest conceptual challenge in turning a 10-second video into a playable physics world is handling time: *once gravity or user interaction takes over, objects leave their original recorded trajectories.*

SceneFork solves this with a **"Timeline Fork" architecture**:

1. **Play Mode ($t = 0$ to t_{fork}):** The video plays normally until the user hits **"Fork Scene"** (or speaks a command) at any frame t_{fork}.
2. **Temporal Sprite Extraction:** Instead of only grabbing a static single frame, `SAM 3.1` tracks every object across the video window to extract both its **collision hull at** t_{fork} and a **looping animated texture strip** (e.g., a pedestrian's walking legs, a spinning wheel, or flickering neon sign) so objects stay visually alive even after physics takes over.
3. **Clean-Plate Separation:** `Muse Image` erases all dynamic foreground objects from frame t_{fork}, generating an unoccluded static background canvas.
4. **Live Physics Takeover:** At t_{fork}, the flat video player seamlessly crossfades in zero milliseconds into an HTML5 **PixiJS + Rapier2D** canvas where every segmented object is now an independent rigid/soft body sitting on top of the clean-plated background.

2. End-to-End Architecture & Meta Model Pipeline

```
[Input MP4 Clip] + [Live Microphone Stream]
       │
       ├──► 1. SAM 3.1 (Modal GPU) ──────────► Alpha Masks + Simplified
Convex Polygons + Sprite Loops
       │         │
       │         ├──► (Dilated Inverse Mask) ─► 2. Muse Image ──► Clean-
Plated Background + Style Skins
       │         │
       │         └──► (Cropped Object + Context) ► 3. Muse Wonder ─► Initial
Physics Manifest (JSON)
       │
       ├──► 4. Muse Voice Transcribe + Muse Glimmer (Baseten) ─► Deictic
Voice Intent + Fast Safety/Rule Router
       │
```

```
        └──► 5. Muse Code (Autonomous Sandbox Agent) ──────────► Hot-
Swappable Rapier2D + WebAudio ECS Patches
```

Stage-by-Stage Model Integration

Stage	Meta Model Used	Exact Input	Exact Output	Why LLMs Here
1. Spatial & Temporal Deconstruction	**SAM 3.1**	Raw 1080p MP4 frames + automatic grid/point prompts at keyframe t_{fork}.	Per-object RGBA sprite cutouts, tracking IDs, and binary mask arrays across frames.	Text/ APIs retur `[x1, x2,` boun boxe caus invisi collis physi engir
2. Background Clean-Plating & Reskinning	**Muse Image**	Keyframe t_{fork} + unified binary mask of all foreground objects dilated by 12 pixels.	Seamless background plate with all movable objects erased + optional stylized sprite textures.	Witho clear pickir parke a vid leave shap hole dupli behir
3. Physical Material Grounding	**Muse Wonder**	Annotated keyframe showing numbered `SAM 3.1` mask overlays + cropped object thumbnails.	Structured `ScenePhysicsManifest` JSON (mass, friction, restitution, body type, anchor joints).	Requ multi physi comr sens (knov stree anch the pave while bask dyna boun

Stage	Meta Model Used	Exact Input	Exact Output	Why LLMs Here
4. Real-Time Deictic Voice Router	**Muse Voice Transcribe** + **Muse Glimmer** (Open Weights on Baseten)	Streaming microphone audio + current cursor `(x, y)` coordinates + active object IDs.	Sub-100ms structured intent packet mapping spoken pronouns (*"make **this** bouncy"*) to exact `SAM 3.1` mask IDs.	Cloud mono take seco parse `Muse Glim` Base route para twea less 90ms
5. World-Rule & Audio Compiler	**Muse Code**	Current `ScenePhysicsManifest` + user voice command + target TypeScript plugin interface.	Executable JavaScript/TypeScript ECS system patch (custom gravity fields, joints, procedural WebAudio synths).	Trans *"turn deliv into a sling shoo explo wate* requi multi- proce code synth

3. Deep-Dive Engineering: Solving the 4 Make-or-Break Bottlenecks

Judges award the **20% Technical Depth** score when you show how you solved real computer vision and systems engineering problems rather than gluing basic API calls together.

Bottleneck 1: Pixel Masks to 60 FPS Physics Polygons

If you feed a raw 2,000-vertex pixel contour from `SAM 3.1` directly into a 2D physics engine (like Rapier2D or Matter.js), the physics solver will crash or drop to 4 FPS due to concave self-intersections.

- **The Solution:** Run a fast OpenCV post-processing step between `SAM 3.1` and `Muse Wonder`:

1. Extract the external contour using `cv2.findContours()`.
2. Simplify the polygon hull using the **Douglas-Peucker algorithm** (`cv2.approxPolyDP`) with $\epsilon = 0.015 \times \text{perimeter}$ to reduce each object to 8–16 clean vertices.
3. Decompose concave shapes into convex sub-polygons (`earcut` / `poly-decomp`) before binding the `SAM 3.1` RGBA sprite texture to the body's centroid.

Python

```
# backend/vision/hull_compiler.py
import cv2
import numpy as np

def mask_to_physics_hull(binary_mask: np.ndarray, epsilon_factor: float =
0.012):
    contours, _ = cv2.findContours(binary_mask, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
    largest = max(contours, key=cv2.contourArea)

    # Calculate centroid for sprite-to-rigidbody alignment
    M = cv2.moments(largest)
    cx = int(M["m10"] / max(M["m00"], 1e-5))
    cy = int(M["m01"] / max(M["m00"], 1e-5))

    # Simplify contour to 8-20 vertices for 60fps Rapier2D collision
    peri = cv2.arcLength(largest, True)
    approx = cv2.approxPolyDP(largest, epsilon_factor * peri, True)

    # Convert vertices to local coordinates relative to centroid
    local_vertices = [[int(pt[0][0] - cx), int(pt[0][1] - cy)] for pt in
approx]
    return {"centroid": [cx, cy], "vertices": local_vertices}
```

Bottleneck 2: Eliminating Shadow & Edge Halos During Clean-Plating

`SAM 3.1` masks are pixel-tight. If you pass an exact m-ask to `Muse Image` for background inpainting, residual compression fringes and cast drop-shadows remain on the background plate, leaving ghostly outlines when objects move.

- **The Solution:** Apply a **15-pixel morphological dilation** (`cv2.dilate` with an elliptical kernel) plus a directional shadow-region heuristic to the union of all dynamic `SAM 3.1` masks before calling `Muse Image`. Keep the *undilated* mask for the foreground sprite cutout, and use the *dilated* mask for the background clean-plate.

Bottleneck 3: Grounding Natural Physics with `Muse Wonder`

When a video is segmented into 12 masks, some objects should fall when poked (a coffee cup, a car), some should act as immovable ground colliders (sidewalks, walls, tables), and some should hinge on a pivot (a swinging shop sign).

- **The System Prompt for `Muse Wonder`:**

JSON

```
{
  "system_prompt": "You are the Physical World Grounding Engine for
SceneFork. Given a keyframe with numbered SAM 3.1 segmentation contours,
output a strict JSON array assigning real-world 2D physics properties to
each mask ID.",
  "schema_example": {
    "gravity": [0.0, 9.81],
    "ground_plane_mask_ids": [1, 4],
    "entities": [
      {
        "mask_id": 2,
        "semantic_label": "red_delivery_van",
        "body_type": "dynamic",
        "mass_kg": 1500.0,
        "restitution": 0.15,
        "friction": 0.8,
        "acoustic_material": "hollow_metal",
        "z_layer": 2
      },
      {
        "mask_id": 5,
        "semantic_label": "hanging_street_sign",
        "body_type": "dynamic",
        "mass_kg": 8.0,
        "restitution": 0.4,
        "friction": 0.3,
        "joint_constraint": {
          "type": "revolute_hinge",
          "anchor_to_mask_id": 3,
          "local_anchor_px": [0, -45]
        }
      }
    ]
  }
}
```

Bottleneck 4: Safe, Zero-Crash Hot-Reloading for `Muse Code`

If `Muse Code` rewrites the entire physics canvas file every time the user speaks a rule, the simulation will reset and risked syntax errors could crash the demo live on stage.

- **The Solution: Sandboxed ECS Rule Plugins + Visual Self-Critique.**
  - The core PixiJS + Rapier2D engine stays locked.
  - `Muse Code` only writes isolated **World Mutator Modules** that conform to a strict TypeScript interface (`onTick(world, entities, dt)`, `onCollision(entityA, entityB, impulse)`, `onPointerDown(entity, pointer)`).
  - **Autonomous Critic Loop (Crucial for the 20% Technical Depth Rubric):** Before injecting a `Muse Code` patch into the live canvas, the engine runs the patch for 30 headless simulation ticks (0.5 seconds of simulated time) in a background WebWorker. If an object's velocity explodes to `NaN` or teleports off-screen, `Muse Glimmer` flags the anomaly and `Muse Code` self-corrects the force multiplier automatically before the user ever sees it.

4. Procedural Audio: The Secret Weapon for the Demo Score

Videos lose their magic when objects collide silently. Instead of using pre-recorded stock sound effects, instruct `Muse Code` to synthesize **WebAudio API procedural collision synths** tied to the `acoustic_material` tag inferred by `Muse Wonder` (`hollow_metal`, `glass`, `wood`, `rubber`) and scaled by the exact collision impulse from Rapier2D:

- When the delivery van (`hollow_metal`) slams into a streetlamp at high velocity, the Rapier2D collision callback triggers a metallic FM synthesis transient whose gain and filter resonance scale with kinetic impact energy.
- When the user says, *"Turn these parked cars into a playable marimba,"* `Muse Code` binds pentatonic scale frequencies to each car's collision callback.

5. Repository Structure & Tech Stack Blueprint

Plaintext

```
scenefork/
├── frontend/                        # Vite + React + TypeScript
│   ├── src/
│   │   ├── engine/
│   │   │   ├── SceneRuntime.ts      # PixiJS v8 renderer + Rapier2D WASM
physics loop
│   │   │   ├── HeadlessVerifier.ts  # Runs Muse Code patches for 30 ticks
to prevent NaN explosions
│   │   │   ├── ProceduralAudio.ts   # WebAudio synthesizer driven by
```

```
collision impulses
│   │   │   └── DeicticTracker.ts    # Tracks mouse (x,y) during voice
commands for "this/that" binding
│   │   ├── components/
│   │   │   ├── VideoForkTimeline.tsx # Scrubbable video player that
crossfades into Canvas at t_fork
│   │   │   ├── MaskInspectorHUD.tsx  # Toggleable neon overlay showing SAM
3.1 hulls & center-of-mass
│   │   │   └── VoiceDirectorBar.tsx  # Live waveform + Muse Voice
Transcribe streaming transcript
├── backend/                         # FastAPI + Python 3.11
│   ├── modal_workers/
│   │   ├── sam31_pipeline.py        # GPU worker: SAM 3.1 video
segmentation + OpenCV hull extraction
│   │   └── glimmer_router.py        # Open-weight Muse Glimmer / Spark Open
fast intent classifier
│   ├── agents/
│   │   ├── wonder_physics.py        # Muse Wonder multimodal prompt chain
for ScenePhysicsManifest
│   │   ├── image_cleanplate.py      # Muse Image inpainting & mask-
constrained style transfer
│   │   └── muse_code_compiler.py    # Muse Code ECS plugin generator + AST
validator
│   └── main.py                      # WebSocket orchestrator streaming
stage updates to frontend
```

6. 48-Hour Hackathon Execution Roadmap

Time Block	Milestone	Key Deliverable
Hours 0–6	**Core Physics Canvas**	Build the PixiJS + Rapier2D frontend taking a mock JSON of 4 sprite cutouts, polygon hulls, and a static background image. Get mouse dragging/throwing working at 60 FPS.
Hours 6–14	**SAM 3.1 + Hull Compiler**	Deploy the `SAM 3.1` pipeline + OpenCV Douglas-Peucker polygon simplifier. Verify that dropping an MP4 extracts clean RGBA sprites and accurate collision hulls.
Hours 14–22	**Muse Image Clean-Plating**	Connect dilated inverse masks from `SAM 3.1` to `Muse Image`. Ensure moving an object out of its starting spot reveals a clean, hole-free background behind it.
Hours 22–30	**Muse Wonder Physics Grounding**	Wire `Muse Wonder` to automatically classify static ground masks vs. dynamic rigid bodies, mass, and bounce so scenes behave realistically with zero manual setup.

Time Block	Milestone	Key Deliverable
Hours 30–40	**Voice + Muse Code Hot-Swap**	Integrate `Muse Voice Transcribe`, `Muse Glimmer` (via Modal/Baseten for fast deictic routing), and `Muse Code` to hot-inject custom gravity, slingshots, and WebAudio synths.
Hours 40–48	**HUD Polish & 3 Golden Clips**	Build the "X-Ray Debug HUD" (lets judges see `SAM 3.1` polygons, center of mass, and live `Muse Code` patches) and pre-test 3 high-impact 10-second video clips.

7. Pre-Planned "3 Golden Clips" for a Bulletproof Live Demo

Never rely on a single random video during a live pitch. Have three contrasting 6-second clips loaded in the sidebar—plus allow a judge to AirDrop a clip from their phone at the end:

1. **Clip 1: Urban Street Scene (The Physics Hook):** A parked food truck, streetlamp, traffic cones, and a passing skateboarder. *Demo action:* Fork the video mid-jump, grab the skateboarder with your mouse, bounce them off the food truck awning like a trampoline, and command via voice: *"Turn the food truck into an Angry Birds slingshot and make the traffic cones explode into confetti on impact."*
2. **Clip 2: Desk / Kitchen Countertop (The Audio & Style Hook):** Five coffee mugs and glasses lined up on a wooden table. *Demo action:* Command via voice: *"Turn these mugs into a playable lo-fi marimba, spawn bouncing rain droplets from the top of the screen, and reskin the mugs into stained glass."*
3. **Clip 3: Live Webcam / Judge Smartphone Capture (The Mic Drop):** Record a 5-second clip of the hackathon venue or your own desk live on camera, hit **Fork Scene**, and immediately toss the physical objects on your desk around inside the browser window.